Functor, Applicative, Monad

licative/Monad · Combine async user+order fetches into a dashboard view · Build

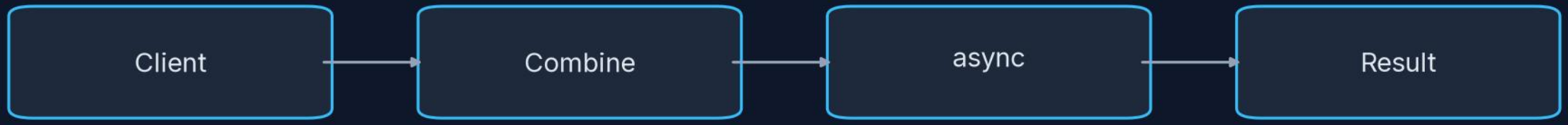
Today's Hook

- Problem: Combine async user+order fetches into a dashboard view
- Pattern: Functor/Applicative/Monad — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Combine async user+order fetches into a dashboard view
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Functor/Applicative/Monad: Combine async user+order fetches into a dashboard

Real-World Scenario

- Running example: Combine async user+order fetches into a dashboard view
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.functional.fam.problem;

/** Naive: nested null checks / sequential blocking gets — no composition. */
public final class NaiveCombine {
    public static AsyncDashboard.Dashboard combine(AsyncDashboard.User u, AsyncDashboard.Order o) {
        if (u == null || o == null) return null;
        return new AsyncDashboard.Dashboard(u.name(), o.totalCents());
    }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.functional.fam.problem;

import java.util.Optional;
import java.util.concurrent.CompletableFuture;
import java.util.function.BiFunction;
import java.util.function.Function;

/**
 * Functor / Applicative / Monad over Optional and CompletableFuture:
 * combine async user + order fetches into a dashboard view.
 */
public final class AsyncDashboard {
    private AsyncDashboard() {}

    public record User(String id, String name) {}
    public record Order(String id, String userId, int totalCents) {}
    public record Dashboard(String name, int totalCents) {}

    // ---- Optional as Functor/Applicative/Monad ----
    public static <A, B> Optional<B> map(Optional<A> fa, Function<A, B> f) { return fa.map(f); }
    public static <A, B> Optional<B> flatMap(Optional<A> fa, Function<A, Optional<B>> f) { return fa.flatMap(f); }
    public static <A, B, C> Optional<C> map2(Optional<A> fa, Optional<B> fb, BiFunction<A, B, C> f) {
        return fa.flatMap(a -> fb.map(b -> f.apply(a, b)));
    }
```

Solution (2)

```
    public static Optional<Dashboard> dashboardOpt(Optional<User> user, Optional<Order> order) {
        return map2(user, order, (u, o) -> new Dashboard(u.name(), o.totalCents()));
    }

    // ---- CompletableFuture applicative-style zip ----
    public static <A, B, C> CompletableFuture<C> map2Future(
            CompletableFuture<A> fa, CompletableFuture<B> fb, BiFunction<A, B, C> f) {
        return fa.thenCombine(fb, f);
    }

    public static CompletableFuture<Dashboard> dashboardAsync(
            CompletableFuture<User> user, CompletableFuture<Order> order) {
        return map2Future(user, order, (u, o) -> new Dashboard(u.name(), o.totalCents()));
    }

    public static CompletableFuture<User> fetchUser(String id, boolean ok) {
        return CompletableFuture.supplyAsync(() -> {
            if (!ok) throw new IllegalStateException("user-missing");
            return new User(id, "User-" + id);
        });
    }
```

Solution (3)

```
    public static CompletableFuture<Order> fetchOrder(String id, String userId, boolean ok) {
        return CompletableFuture.supplyAsync(() -> {
            if (!ok) throw new IllegalStateException("order-missing");
            return new Order(id, userId, 4200);
        });
    }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.functional.fam.problem;

import java.util.Optional;
import java.util.concurrent.CompletionException;

public final class ProblemDemo {
    public static void main(String[] args) throws Exception {
        System.out.println("=== Functor/Applicative/Monad: async dashboard ===");
        System.out.println("--- naive null combine ---");
        System.out.println(NaiveCombine.combine(new AsyncDashboard.User("1", "Ada"), null));

        System.out.println("--- Optional map2 ---");
        System.out.println(AsyncDashboard.dashboardOpt(
                Optional.of(new AsyncDashboard.User("1", "Ada")),
                Optional.of(new AsyncDashboard.Order("o1", "1", 4200))));
        System.out.println(AsyncDashboard.dashboardOpt(Optional.of(new AsyncDashboard.User("1", "Ada")), Optional.empty()));

        System.out.println("--- CompletableFuture map2 ---");
        var dash = AsyncDashboard.dashboardAsync(
                AsyncDashboard.fetchUser("1", true),
                AsyncDashboard.fetchOrder("o1", "1", true)).join();
        System.out.println(dash);
```

Runnable main (2)

```
        System.out.println("--- failure path ---");
        try {
            AsyncDashboard.dashboardAsync(
                    AsyncDashboard.fetchUser("1", true),
                    AsyncDashboard.fetchOrder("o1", "1", false)).join();
        } catch (CompletionException e) {
            System.out.println("caught=" + e.getCause().getMessage());
        }
    }
}
```

Walkthrough Output

```
=== Functor/Applicative/Monad: async dashboard ===
--- naive null combine ---
null
--- Optional map2 ---
Optional[Dashboard[name=Ada, totalCents=4200]]
Optional.empty
--- CompletableFuture map2 ---
Dashboard[name=User-1, totalCents=4200]
--- failure path ---
caught=order-missing
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Functor/Applicative/Monad.

Related Patterns

- Contrast Functor/Applicative/Monad with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Functor/Applicative/Monad → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.